Method Chaining

(Also available in Pyret)

Students learn how to chain Transformers together, and define more sophisticated subsets.

Lesson Goals	Students will be able to… • Use chaining to write more sophisticated analyses using fewer clicks • Identify bugs introduced by chaining Transformers in the wrong order
Student-facing Lesson Goals	• Let's practice combining Transformers together.
Prerequisites	• Introduction to Data Science • Exploring CODAP • Bar Charts • Introduction to Transformers: Sort • More Transformers: Filter, Transform, Build
Materials	• Lesson Slides • Animals Starter File • Design Recipe (Page 23) • Design Recipe 2 (Page 24) • Chaining (Page 25) • Chaining 2: Order Matters (Page 26) • *Optional: Table Transformations with Chaining*
Preparation	• All students should log into CODAP and open their saved "Animals Starter File". If they don't have the file, they can open a new one from Animals Starter File.

Design Recipe Practice *25 minutes*

Overview

In this lesson students build on what they have already learned to formalize their understanding of the Design Recipe, and then gain fluency with using the Design Recipe.

Launch

The Design Recipe is a sequence of steps that helps us document, test out, and write the expressions that our Transformers need to work properly - enabling us to thoughtfully analyze data! It's important for this to be like second nature, so let's get some practice using it.

- Let's look at this Design Recipe worksheet together.

First, we need to decide which Transformer to use: Filter, Transform, or Build. Based on the Transformer's already-provided name, students should deduce that they will use Filter. They can then record the Transformer's name on the line.

Transformer (check one)	Filter	Transform	Build

filter-is-dog

Transformer name

Next, we provide **example tables** . In this case, we want to know the animals' names and their species, so we write down those column names. We want to list a few different animals - at least one that is a dog, and at least one that is not - to represent the variety of animals on the table. Then we think about what our transformed table will look like:

- Will Sasha be on the new table? *No, Sasha is a cat. We only want dogs!*
- Will Fritz be on the new table? *Yes, Fritz is a dog.*
- Will Toggle be on the new table? *Yes, Toggle is a dog.*

Example Tables

What gets filtered/transformed/built? In the sample tables below, add the relevant columns.

Original Table

name	species
sasha	cat
fritz	dog
toggle	dog

Transformed Table

name	species
fritz	dog
toggle	dog

Now, we are ready to drill down on the contents of our Transformer.

- First - the **contract** , which requires a domain (what type of data will we provide) and a range (what type of data will be produced). Whenever we are filtering, we can expect the contract to be the same: Row -> Boolean.
- Next, we need a clear **purpose statement** which describes what the expression does to each row. In this case, the expression will consume an animal and compute whether the species is "dog" - as our example tables (above) demonstrate!
- And finally, we enter our **expression** , in this case: `species = "dog"` .

Contents (Contract, Purpose Statement, and Expression)

Row	->	Boolean
Domain		*Range*

consumes an animal and computes whether the species is "dog"

Purpose: what does the formula do for each row?

species = "dog"

i.e. Weight < 20 or Species = "rabbit". Pay careful attention to capitalization and quotation marks.

Each time students encounter a new word problem, we encourage working through it with paper and pencil, as above; the Design Recipe slows down students' thinking and encourages them to reason through each scenario fully rather than guessing haphazardly.

Investigate

- Open your saved Animals Starter File.
- Define the Transformers and expressions on Design Recipe 2.

Optional: Combining Booleans

Suppose we want to build a table of Animals that are fixed *and* old, or a table of animals that are cats *or* dogs?

By using the **`and`** and **`or`** operators, we can *combine* Boolean tests on a single Transformer, like `Filter`. Once we've opened the `Filter` Transformer, we would tell CODAP to keep all rows that satisfy `Species = "cat"` **`and`** `Species = "dog"`. This is handy for more complex programs! For example, we might want to ask if a character in a video game has run out of health points *and* if they have any more lives. We might want to know if someone's ZIP Code puts them in Texas or New Mexico. When you go out to eat at a restaurant, you might ask what items on the menu have meat and cheese.

When we want to chain together *different* Transformers, however, this strategy will not work. We'll need to find another way!

Synthesize

- Did you find yourselves getting faster at using the Design Recipe?
- What patterns or shortcuts are you noticing, when you use the Design Recipe?

Chaining *25 minutes*

Overview

Students learn how to compose multiple table operations (sorting, filtering, building) on the same table - a technique called "chaining".

Launch

A journalist comes to the shelter who wants to write a story about a successful pet adoption — but she has a very specific set of criteria. The reporter wants to report on the adoption of an animal that weighs **no more than 9 kilograms.** She also wants to review an updated copy of the dataset each week (reflecting changes to the shelter's population) before making a decision about which animal to showcase. To help the journalist, you decide that you want to define and *chain together* two Transformers - *Build Attribute* and *Filter*.

- We'll define a ***Build Attribute*** Transformer using the formula `pounds/2.205`. (CODAP also requires that we provide a name for our new attribute, such as `kilograms`.)
- We'll define a ***Filter*** Transformer using the formula `kilograms<9`.

- Which of the two above Transformers should we apply *first*, `weight-in-kg` or `filter-if-light`?
  - *We must apply* `weight-in-kg` *first. (*`Kilograms` *doesn't exist on our table otherwise!)*
- What do you predict will happen if we apply them in the wrong order? Why?
  - *Students should predict that they will encounter some sort of error message.*

A perk of applying Transformers, rather than manipulating the dataset, is that any updates made to the original dataset will flow through the chain. As you chain together Transformers, it is important to remember the following:

Order matters: Build / Transform, Filter, Sort.

Suppose we want to build a column and then use it to filter our table. If we use the Transformers in the wrong order (trying to filter by a column that doesn't exist yet), we might wind up crashing the program. Even worse, the program might work, but produce results that are incorrect!

Tip: Saving Transformers and Renaming Tables

Saving a particular configuration of a Transformer is useful so that the Transformer can be easily accessed in the future. When we save a Transformer, we want to give it a useful name and purpose statement for ease of use later.
We also encourage students to rename tables descriptively. By the end of this exercise, the table students create will have quite a lengthy name: `(weight-in-kg(filter-if-light(Animals-Dataset)))`. That's a lot of parentheses! As an alternative, students might consider using renaming the table. For instance, `light-animals-in-kg` might be a more useful table name, here.

Investigate

- How well do you know your Transformers?
- Complete Chaining and Chaining 2: Order Matters in to find out.

Synthesize

Ask students about their answers to Chaining 2: Order Matters.

- Which ones produced an error?
- Why will they produce an error?
- How could they be fixed?

Can students trigger a similar error in their Table Methods starter file?
As our analysis gets more complex, chaining is a great way to re-use code we've already written. And less code means a smaller chance of bugs.

Additional Exercises

- Table Transformations with Chaining